Design and Analysis of Algorithm (DAA), Lab-11

BTech (CS-B and CE), 3rd Semester
October 27, 2026



Instructor: Dr. Ajaya Kumar Dash.

Questions based on the applications of Breadth-First-Search (BFS) algorithm

1. **[Shortest Path on Parity-Constrained Graphs via 0-1 BFS]** You are given a directed graph $G = (V, E)$, a designated source vertex $S \in V$, and an extra penalty cost $C \geq 0$. Every directed edge $(u, v) \in E$ has a weight $w(u, v) \in \{0, 1\}$.

   For any path P from S to a vertex v:

   1. Let $W(P)$ denote the sum of all edge weights along P.
   2. Let $k_1(P)$ denote the total count of weight-1 edges traversed in P.
   3. The effective cost of path P is defined as:

$$\text{Cost}(P) = W(P) + \begin{cases} 0, & \text{if } k_1(P) \text{ is even} \\ C, & \text{if } k_1(P) \text{ is odd} \end{cases}$$

   Objective: For every vertex $v \in V$, compute the minimum possible Cost(P) among all valid paths from S to v. If a node v cannot be reached from S, report that it is unreachable. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

2. **[Multi-Source BFS for Radius-Constrained Voronoi Cell Partitioning]** You are given:

   - An unweighted, undirected graph $G = (V, E)$.
   - A set of K designated hub vertices $H = \{h_1, h_2, \ldots, h_K\} \subseteq V$, indexed from 1 to K.
   - An integer radius threshold $R \geq 0$.

   Let dist(u, v) denote the shortest path distance (minimum number of edges) between vertices u and v in G.

   For every vertex $v \in V$, assign an ownership label owner$[v] \in \{1, 2, \ldots, K\} \cup \{-1\}$ according to the following rules:

   1. *Distance Metric*: Find the distance from v to each service hub: $d_i = \text{dist}(v, h_i)$ for all $i \in \{1, \ldots, K\}$. Let $d_{\min}(v) = \min_{1 \leq i \leq K} d_i$.
   2. *Reachability Threshold (R)*: If $d_{\min}(v) > R$, vertex v lies outside the service coverage area. Set owner$[v] = -1$.
   3. *Closest Hub Assignment*: If $d_{\min}(v) \leq R$, assign v to the hub h_i that minimizes d_i.

4. *Deterministic Tie-Breaking*: If multiple hubs achieve the exact same minimum distance $d_{\min}(v)$, choose the hub with the smallest numerical index i.

Objective: Output the array owner$[1 \dots |V|]$ containing the assigned hub index (or -1 if unassigned) and the corresponding shortest distance to that hub for every vertex. By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.

3. **[Bidirectional BFS for Shortest Transformation Sequence in a Word Graph]** You are given:

   - A dictionary D containing N unique words, where every word has the exact same length L.
   - A starting word **A** and a target word **B**, both of length L.

   A *valid mutation step* consists of changing exactly one character of a word such that the resulting word exists in dictionary D. A **transformation sequence** from **A** to **B** is an ordered sequence of words:

   $$W_0, W_1, W_2, \dots, W_k$$

   such that:

   1. $W_0 = \mathbf{A}$ and $W_k = \mathbf{B}$.
   2. Every intermediate word $W_i \in D$ for all $1 \leq i \leq k$. (Note: whether **A** must initially be in D is typically specified by the input constraints, but **B** must always be in D.)
   3. For every adjacent pair (W_{i-1}, W_i), the two words differ by exactly one character (i.e., their Hamming distance is 1).

   Objective: Compute the minimum number of mutations k required to transform **A** into **B**. If **A** cannot be transformed into **B** through valid intermediate words in D, return -1 (or report that no valid path exists). Keep in mind to consider the following constraints for this problem during implementation:

   - $N \leq 10^5$ (number of dictionary words)
   - $L \leq 10$ (length of each word)
   - All words consist solely of lowercase English letters ('a' to 'z').

   By choosing the proper input representation, write a program in C to validate your algorithm and derive the complexity analysis of your algorithm.